

Support vector machine

In machine learning, a **support vector machine** (SVM) or **support vector network**^[1] is a supervised max-margin model with associated learning algorithms that analyze data for classification and regression analysis. Developed at AT&T Bell Laboratories,^{[1][2]} SVMs are one of the most studied models, being based on statistical learning frameworks of VC theory proposed by Vapnik (1982, 1995) and Chervonenkis (1974).

In addition to performing linear classification, SVMs can efficiently perform non-linear classification using the kernel trick, representing the data only through a set of pairwise similarity comparisons between the original data points using a kernel function, thereby transforming them into coordinates in a higher-dimensional feature space. Thus, SVMs use the kernel trick to implicitly map their inputs into high-dimensional feature spaces, where linear classification can be performed. Furthermore, the computational complexity arising from using the kernel trick warrants lesser usage of resources.^[3] As members of the max-margin models, SVMs are resilient to noisy data (e.g., misclassified examples). They can also be used for regression tasks, where the objective becomes ϵ -sensitive.

The support vector clustering^[4] algorithm, created by Hava Siegelmann and Vladimir Vapnik, applies the statistics of support vectors, developed in the support vector machines algorithm, to categorize unlabelled data. These data sets require unsupervised learning approaches, which attempt to find natural clustering of the data into groups, and then to map new data according to these clusters.

The popularity of SVMs is likely due to their amenability to theoretical analysis and their flexibility in being applied to a wide variety of tasks, including structured prediction problems. It is not clear that SVMs have better predictive performance than other linear models, such as logistic regression and linear regression.^[5]

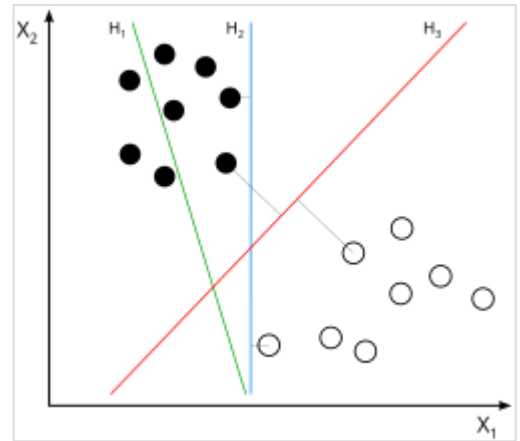
Motivation

Classifying data is a common task in machine learning. Say we have a binary classification problem: suppose some given data points each belong to one of two classes, and the goal is to decide which class a new data point will be in. In the case of support vector machines, a data point is viewed as a p -dimensional vector (a list of p numbers), and we want to know whether we can separate such points with a $(p - 1)$ -dimensional hyperplane. This is called a linear classifier. There are many hyperplanes that might classify the data. One reasonable choice as the best hyperplane is the one that represents the largest separation, or margin, between the two classes. Therefore, we choose a hyperplane such that its distance from the data points nearest to it on each side is maximized. If such a hyperplane exists, it is known as the maximum-margin hyperplane and the linear classifier it defines is known as a maximum-margin classifier; or equivalently, the perceptron of optimal stability.^[6]

More formally, a support vector machine constructs a hyperplane or set of hyperplanes in a high or infinite-dimensional space, which can be used for classification, regression, or other tasks like outlier detection.^[7] Intuitively, a good separation is achieved by the hyperplane that has the largest distance to

the nearest training-data point of any class (so-called functional margin), since in general the larger the margin, the lower the generalization error of the classifier.^[8] A lower generalization error means that the implementer is less likely to experience overfitting.

Whereas the original problem may be stated in a finite-dimensional space, it often happens that the sets to discriminate are not linearly separable in that space. For this reason, it was proposed^[9] that the original finite-dimensional space be mapped into a much higher-dimensional space, presumably making the separation easier in that space. To keep the computational load reasonable, the mappings used by SVM schemes are designed to ensure that dot products of pairs of input data vectors may be computed easily in terms of the variables in the original space, by defining them in terms of a kernel function $k(\mathbf{x}, \mathbf{y})$ selected to suit the problem.^[10] The hyperplanes in the higher-dimensional space are defined as the set of points whose dot product with a vector in that space is constant, where such a set of vectors is an orthogonal (and thus minimal) set of vectors that defines a hyperplane. The vectors defining the hyperplanes can be chosen to be linear combinations with parameters α_i of images of feature vectors $\mathbf{x}_i$ that occur in the data base. With this choice of a hyperplane, the points $\mathbf{x}$ in the feature space that are mapped into the hyperplane are defined by the relation $\sum_i \alpha_i k(\mathbf{x}_i, \mathbf{x}) = \text{constant}$. Note that if $k(\mathbf{x}, \mathbf{y})$ becomes small as $\mathbf{y}$ grows further away from $\mathbf{x}$, each term in the sum measures the degree of closeness of the test point $\mathbf{x}$ to the corresponding data base point $\mathbf{x}_i$. In this way, the sum of kernels above can be used to measure the relative nearness of each test point to the data points originating in one or the other of the sets to be discriminated. Note the fact that the set of points $\mathbf{x}$ mapped into any hyperplane can be quite convoluted as a result, allowing much more complex discrimination between sets that are not convex at all in the original space.



H_1 does not separate the classes. H_2 does, but only with a small margin. H_3 separates them with the maximal margin.

Applications

SVMs can be used to solve various real-world problems:

- SVMs are helpful in text and hypertext categorization, as their application can significantly reduce the need for labeled training instances in both the standard inductive and transductive settings.^[11] Some methods for shallow semantic parsing are based on support vector machines.^[12]
- Classification of images can also be performed using SVMs. Experimental results show that SVMs achieve significantly higher search accuracy than traditional query refinement schemes after just three to four rounds of relevance feedback. This is also true for image segmentation systems, including those using a modified version SVM that uses the privileged approach as suggested by Vapnik.^{[13][14]}
- Classification of satellite data like SAR data using supervised SVM.^[15]
- Hand-written characters can be recognized using SVM.^{[16][17]}
- The SVM algorithm has been widely applied in the biological and other sciences. They have been used to classify proteins with up to 90% of the compounds classified correctly. Permutation tests based on SVM weights have been suggested as a mechanism for

interpretation of SVM models.^{[18][19]} Support vector machine weights have also been used to interpret SVM models in the past.^[20] Posthoc interpretation of support vector machine models in order to identify features used by the model to make predictions is a relatively new area of research with special significance in the biological sciences.

History

The original SVM algorithm was invented by [Vladimir N. Vapnik](#) and [Alexey Ya. Chervonenkis](#) in 1964. In 1992, [Bernhard Boser](#), [Isabelle Guyon](#) and [Vladimir Vapnik](#) suggested a way to create nonlinear classifiers by applying the [kernel trick](#) to maximum-margin hyperplanes.^[9] The "soft margin" incarnation, as is commonly used in software packages, was proposed by [Corinna Cortes](#) and Vapnik in 1993 and published in 1995.^[1]

Linear SVM

We are given a training dataset of n points of the form

$$(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n),$$

where the y_i are either 1 or -1, each indicating the class to which the point $\mathbf{x}_i$ belongs. Each $\mathbf{x}_i$ is a p -dimensional [real](#) vector. We want to find the "maximum-margin hyperplane" that divides the group of points $\mathbf{x}_i$ for which $y_i = 1$ from the group of points for which $y_i = -1$, which is defined so that the distance between the hyperplane and the nearest point $\mathbf{x}_i$ from either group is maximized.

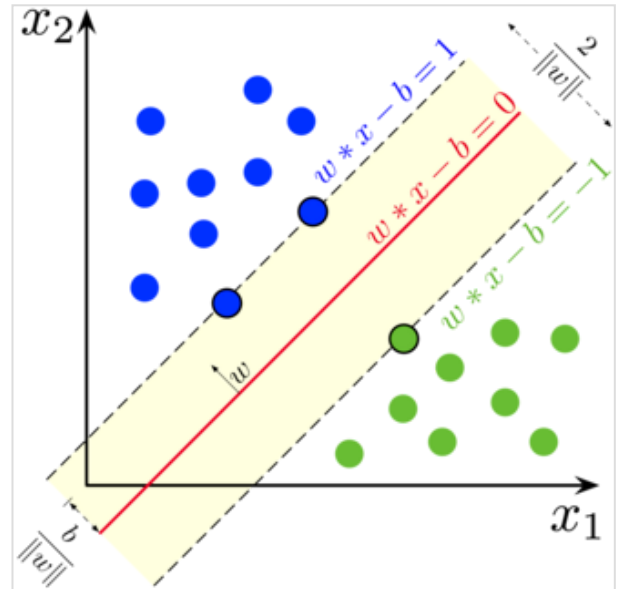
Any [hyperplane](#) can be written as the set of points $\mathbf{x}$ satisfying

$$\mathbf{w}^T \mathbf{x} - b = 0,$$

where $\mathbf{w}$ is the (not necessarily normalized) [normal vector](#) to the hyperplane. This is much like [Hesse normal form](#), except that $\mathbf{w}$ is not necessarily a unit vector. The parameter $\frac{b}{\|\mathbf{w}\|}$ determines the offset of the hyperplane from the origin along the normal vector $\mathbf{w}$.

The bias may also be defined so that

$$\mathbf{w}^T \mathbf{x} + b = 0.$$



Maximum-margin hyperplane and margins for an SVM trained with samples from two classes. Samples on the margin are called the support vectors. □

Hard-margin

If the training data is linearly separable, we can select two parallel hyperplanes that separate the two classes of data, so that the distance between them is as large as possible. The region bounded by these two hyperplanes is called the "margin", and the maximum-margin hyperplane is the hyperplane that lies halfway between them. With a normalized or standardized dataset, these hyperplanes can be described by the equations

$$\mathbf{w}^T \mathbf{x} - b = 1 \text{ (anything on or above this boundary is of one class, with label 1)}$$

and

$$\mathbf{w}^T \mathbf{x} - b = -1 \text{ (anything on or below this boundary is of the other class, with label -1).}$$

Geometrically, the distance between these two hyperplanes is $\frac{2}{\|\mathbf{w}\|}$,^[21] so to maximize the distance between the planes we want to minimize $\|\mathbf{w}\|$. The distance is computed using the distance from a point to a plane equation. We also have to prevent data points from falling into the margin, we add the following constraint: for each i either

$$\mathbf{w}^T \mathbf{x}_i - b \geq 1, \text{ if } y_i = 1,$$

or

$$\mathbf{w}^T \mathbf{x}_i - b \leq -1, \text{ if } y_i = -1.$$

These constraints state that each data point must lie on the correct side of the margin.

This can be rewritten as

$$y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1, \quad \text{for all } 1 \leq i \leq n. \tag{1}$$

We can put this together to get the optimization problem:

$$\begin{aligned} &\underset{\mathbf{w}, b}{\text{minimize}} && \frac{1}{2} \|\mathbf{w}\|^2 \\ &\text{subject to} && y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1 \quad \forall i \in \{1, \dots, n\} \end{aligned}$$

The $\mathbf{w}$ and b that solve this problem determine the final classifier, $\mathbf{x} \mapsto \text{sgn}(\mathbf{w}^T \mathbf{x} - b)$, where $\text{sgn}(\cdot)$ is the sign function.

An important consequence of this geometric description is that the max-margin hyperplane is completely determined by those $\mathbf{x}_i$ that lie nearest to it (explained below). These $\mathbf{x}_i$ are called *support vectors*.

Soft-margin

To extend SVM to cases in which the data are not linearly separable, the hinge loss function is helpful

$$\max (0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i - b)) .$$

Note that y_i is the i -th target (i.e., in this case, 1 or -1), and $\mathbf{w}^\top \mathbf{x}_i - b$ is the i -th output.

This function is zero if the constraint in [\(1\)](#) is satisfied, in other words, if $\mathbf{x}_i$ lies on the correct side of the margin. For data on the wrong side of the margin, the function's value is proportional to the distance from the margin.

The goal of the optimization then is to minimize:

$$\|\mathbf{w}\|^2 + C \left[\frac{1}{n} \sum_{i=1}^n \max (0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i - b)) \right] ,$$

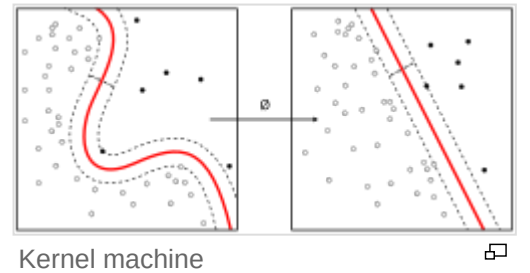
where the parameter $C > 0$ determines the trade-off between increasing the margin size and ensuring that the $\mathbf{x}_i$ lie on the correct side of the margin (Note we can add a weight to either term in the equation above). By deconstructing the hinge loss, this optimization problem can be formulated into the following:

$$\begin{aligned} &\underset{\mathbf{w}, b, \zeta}{\text{minimize}} && \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \zeta_i \\ &\text{subject to} && y_i(\mathbf{w}^\top \mathbf{x}_i - b) \geq 1 - \zeta_i, \quad \zeta_i \geq 0 \quad \forall i \in \{1, \dots, n\} \end{aligned}$$

Thus, for large values of C , it will behave similar to the hard-margin SVM, if the input data are linearly classifiable, but will still learn if a classification rule is viable or not.

Nonlinear kernels

The original maximum-margin hyperplane algorithm proposed by Vapnik in 1963 constructed a linear classifier. However, in 1992, Bernhard Boser, Isabelle Guyon and Vladimir Vapnik suggested a way to create nonlinear classifiers by applying the kernel trick (originally proposed by Aizerman et al.^[22]) to maximum-margin hyperplanes.^[9] The kernel trick, where dot products are replaced by kernels, is easily derived in the dual representation of the SVM problem. This allows the algorithm to fit the maximum-margin hyperplane in a transformed feature space. The transformation may be nonlinear and the transformed space high-dimensional; although the classifier is a hyperplane in the transformed feature space, it may be nonlinear in the original input space.



It is noteworthy that working in a higher-dimensional feature space increases the generalization error of support vector machines, although given enough samples the algorithm still performs well.^[23]

Some common kernels include:

- Polynomial (homogeneous): $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j)^d$. Particularly, when $d = 1$, this becomes the linear kernel.
- Polynomial (inhomogeneous): $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i \cdot \mathbf{x}_j + r)^d$.
- Gaussian radial basis function: $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ for $\gamma > 0$. Sometimes parametrized using $\gamma = 1/(2\sigma^2)$.
- Sigmoid function (Hyperbolic tangent): $k(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i \cdot \mathbf{x}_j + c)$ for some (not every) $\kappa > 0$ and $c < 0$.

The kernel is related to the transform $\varphi(\mathbf{x}_i)$ by the equation $k(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)$. The value $\mathbf{w}$ is also in the transformed space, with $\mathbf{w} = \sum_i \alpha_i y_i \varphi(\mathbf{x}_i)$. Dot products with $\mathbf{w}$ for classification can again be computed by the kernel trick, i.e. $\mathbf{w} \cdot \varphi(\mathbf{x}) = \sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x})$.

Computing the SVM classifier

Computing the (soft-margin) SVM classifier amounts to minimizing an expression of the form

$$\left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i - b)) \right] + \lambda \|\mathbf{w}\|^2. \quad (2)$$

We focus on the soft-margin classifier since, as noted above, choosing a sufficiently small value for λ yields the hard-margin classifier for linearly classifiable input data. The classical approach, which involves reducing (2) to a quadratic programming problem, is detailed below. Then, more recent approaches such as sub-gradient descent and coordinate descent will be discussed.

Primal

Minimizing (2) can be rewritten as a constrained optimization problem with a differentiable objective function in the following way.

For each $i \in \{1, \dots, n\}$ we introduce a variable $\zeta_i = \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i - b))$. Note that ζ_i is the smallest nonnegative number satisfying $y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1 - \zeta_i$.

Thus we can rewrite the optimization problem as follows

$$\begin{aligned} & \text{minimize } \frac{1}{n} \sum_{i=1}^n \zeta_i + \lambda \|\mathbf{w}\|^2 \\ & \text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1 - \zeta_i \text{ and } \zeta_i \geq 0, \text{ for all } i. \end{aligned}$$

This is called the *primal* problem.

Dual

By solving for the Lagrangian dual of the above problem, one obtains the simplified problem

$$\begin{aligned} \text{maximize } f(c_1 \dots c_n) &= \sum_{i=1}^n c_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i c_i (\mathbf{x}_i^\top \mathbf{x}_j) y_j c_j, \\ \text{subject to } \sum_{i=1}^n c_i y_i &= 0, \text{ and } 0 \leq c_i \leq \frac{1}{2n\lambda} \text{ for all } i. \end{aligned}$$

This is called the *dual* problem. Since the dual maximization problem is a quadratic function of the $\mathbf{c}_i$ subject to linear constraints, it is efficiently solvable by quadratic programming algorithms.

Here, the variables $\mathbf{c}_i$ are defined such that

$$\mathbf{w} = \sum_{i=1}^n c_i y_i \mathbf{x}_i.$$

Moreover, $\mathbf{c}_i = 0$ exactly when $\mathbf{x}_i$ lies on the correct side of the margin, and $0 < \mathbf{c}_i < (2n\lambda)^{-1}$ when $\mathbf{x}_i$ lies on the margin's boundary. It follows that $\mathbf{w}$ can be written as a linear combination of the support vectors.

The offset, b , can be recovered by finding an $\mathbf{x}_i$ on the margin's boundary and solving

$$y_i(\mathbf{w}^\top \mathbf{x}_i - b) = 1 \iff b = \mathbf{w}^\top \mathbf{x}_i - y_i.$$

(Note that $y_i^{-1} = y_i$ since $y_i = \pm 1$.)

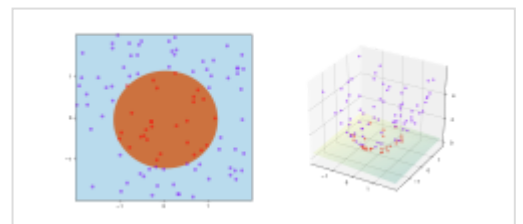
Kernel trick

Suppose now that we would like to learn a nonlinear classification rule which corresponds to a linear classification rule for the transformed data points $\varphi(\mathbf{x}_i)$. Moreover, we are given a kernel function k which satisfies $k(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)$.

We know the classification vector $\mathbf{w}$ in the transformed space satisfies

$$\mathbf{w} = \sum_{i=1}^n c_i y_i \varphi(\mathbf{x}_i),$$

where, the $\mathbf{c}_i$ are obtained by solving the optimization problem



A training example of SVM with kernel $\square$ given by $\varphi((a, b)) = (a, b, a^2 + b^2)$

$$\begin{aligned}
\text{maximize } f(c_1 \dots c_n) &= \sum_{i=1}^n c_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i c_i (\varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)) y_j c_j \\
&= \sum_{i=1}^n c_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i c_i k(\mathbf{x}_i, \mathbf{x}_j) y_j c_j \\
\text{subject to } \sum_{i=1}^n c_i y_i &= 0, \text{ and } 0 \leq c_i \leq \frac{1}{2n\lambda} \text{ for all } i.
\end{aligned}$$

The coefficients c_i can be solved for using quadratic programming, as before. Again, we can find some index i such that $0 < c_i < (2n\lambda)^{-1}$, so that $\varphi(\mathbf{x}_i)$ lies on the boundary of the margin in the transformed space, and then solve

$$\begin{aligned}
b = \mathbf{w}^\top \varphi(\mathbf{x}_i) - y_i &= \left[\sum_{j=1}^n c_j y_j \varphi(\mathbf{x}_j) \cdot \varphi(\mathbf{x}_i) \right] - y_i \\
&= \left[\sum_{j=1}^n c_j y_j k(\mathbf{x}_j, \mathbf{x}_i) \right] - y_i.
\end{aligned}$$

Finally,

$$\mathbf{z} \mapsto \text{sgn}(\mathbf{w}^\top \varphi(\mathbf{z}) - b) = \text{sgn} \left(\left[\sum_{i=1}^n c_i y_i k(\mathbf{x}_i, \mathbf{z}) \right] - b \right).$$

Modern methods

Recent algorithms for finding the SVM classifier include sub-gradient descent and coordinate descent. Both techniques have proven to offer significant advantages over the traditional approach when dealing with large, sparse datasets—sub-gradient methods are especially efficient when there are many training examples, and coordinate descent when the dimension of the feature space is high.

Sub-gradient descent

Sub-gradient descent algorithms for the SVM work directly with the expression

$$f(\mathbf{w}, b) = \left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i - b)) \right] + \lambda \|\mathbf{w}\|^2.$$

Note that f is a convex function of $\mathbf{w}$ and b . As such, traditional gradient descent (or SGD) methods can be adapted, where instead of taking a step in the direction of the function's gradient, a step is taken in the direction of a vector selected from the function's sub-gradient. This approach has the advantage that, for

certain implementations, the number of iterations does not scale with n , the number of data points.^[24]

Coordinate descent

Coordinate descent algorithms for the SVM work from the dual problem

$$\begin{aligned} \text{maximize } f(c_1 \dots c_n) &= \sum_{i=1}^n c_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i c_i (x_i \cdot x_j) y_j c_j, \\ \text{subject to } \sum_{i=1}^n c_i y_i &= 0, \text{ and } 0 \leq c_i \leq \frac{1}{2n\lambda} \text{ for all } i. \end{aligned}$$

For each $i \in \{1, \dots, n\}$, iteratively, the coefficient c_i is adjusted in the direction of $\partial f / \partial c_i$. Then, the resulting vector of coefficients $(c'_1, \dots, c'_n)$ is projected onto the nearest vector of coefficients that satisfies the given constraints. (Typically Euclidean distances are used.) The process is then repeated until a near-optimal vector of coefficients is obtained. The resulting algorithm is extremely fast in practice, although few performance guarantees have been proven.^[25]

Empirical risk minimization

The soft-margin support vector machine described above is an example of an empirical risk minimization (ERM) algorithm for the hinge loss. Seen this way, support vector machines belong to a natural class of algorithms for statistical inference, and many of its unique features are due to the behavior of the hinge loss. This perspective can provide further insight into how and why SVMs work, and allow us to better analyze their statistical properties.

Risk minimization

In supervised learning, one is given a set of training examples $X_1 \dots X_n$ with labels $y_1 \dots y_n$, and wishes to predict y_{n+1} given X_{n+1} . To do so one forms a hypothesis, f , such that $f(X_{n+1})$ is a "good" approximation of y_{n+1} . A "good" approximation is usually defined with the help of a loss function, $\ell(y, z)$, which characterizes how bad z is as a prediction of y . We would then like to choose a hypothesis that minimizes the expected risk:

$$\varepsilon(f) = \mathbb{E} [\ell(y_{n+1}, f(X_{n+1}))].$$

In most cases, we don't know the joint distribution of X_{n+1}, y_{n+1} outright. In these cases, a common strategy is to choose the hypothesis that minimizes the *empirical risk*:

$$\hat{\varepsilon}(f) = \frac{1}{n} \sum_{k=1}^n \ell(y_k, f(X_k)).$$

Under certain assumptions about the sequence of random variables $\mathbf{X}_k, \mathbf{y}_k$ (for example, that they are generated by a finite Markov process), if the set of hypotheses being considered is small enough, the minimizer of the empirical risk will closely approximate the minimizer of the expected risk as n grows large. This approach is called *empirical risk minimization*, or ERM.

Regularization and stability

In order for the minimization problem to have a well-defined solution, we have to place constraints on the set $\mathcal{H}$ of hypotheses being considered. If $\mathcal{H}$ is a normed space (as is the case for SVM), a particularly effective technique is to consider only those hypotheses f for which $\|f\|_{\mathcal{H}} < k$. This is equivalent to imposing a *regularization penalty* $\mathcal{R}(f) = \lambda_k \|f\|_{\mathcal{H}}$, and solving the new optimization problem

$$\hat{f} = \arg \min_{f \in \mathcal{H}} \hat{\varepsilon}(f) + \mathcal{R}(f).$$

This approach is called Tikhonov regularization.

More generally, $\mathcal{R}(f)$ can be some measure of the complexity of the hypothesis f , so that simpler hypotheses are preferred.

SVM and the hinge loss

Recall that the (soft-margin) SVM classifier $\hat{\mathbf{w}}, b : \mathbf{x} \mapsto \text{sgn}(\hat{\mathbf{w}}^T \mathbf{x} - b)$ is chosen to minimize the following expression:

$$\left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x} - b)) \right] + \lambda \|\mathbf{w}\|^2.$$

In light of the above discussion, we see that the SVM technique is equivalent to empirical risk minimization with Tikhonov regularization, where in this case the loss function is the hinge loss

$$\ell(y, z) = \max(0, 1 - yz).$$

From this perspective, SVM is closely related to other fundamental classification algorithms such as regularized least-squares and logistic regression. The difference between the three lies in the choice of loss function: regularized least-squares amounts to empirical risk minimization with the square-loss, $\ell_{sq}(y, z) = (y - z)^2$; logistic regression employs the log-loss,

$$\ell_{\log}(y, z) = \ln(1 + e^{-yz}).$$

Target functions

The difference between the hinge loss and these other loss functions is best stated in terms of *target functions* - the function that minimizes expected risk for a given pair of random variables $\mathbf{X}$, $\mathbf{y}$.

In particular, let $\mathbf{y}_x$ denote $\mathbf{y}$ conditional on the event that $\mathbf{X} = \mathbf{x}$. In the classification setting, we have:

$$\mathbf{y}_x = \begin{cases} 1 & \text{with probability } p_x \\ -1 & \text{with probability } 1 - p_x \end{cases}$$

The optimal classifier is therefore:

$$f^*(x) = \begin{cases} 1 & \text{if } p_x \geq 1/2 \\ -1 & \text{otherwise} \end{cases}$$

For the square-loss, the target function is the conditional expectation function, $f_{sq}(x) = \mathbb{E}[\mathbf{y}_x]$; For the logistic loss, it's the logit function, $f_{log}(x) = \ln(p_x/(1 - p_x))$. While both of these target functions yield the correct classifier, as $\text{sgn}(f_{sq}) = \text{sgn}(f_{log}) = f^*$, they give us more information than we need. In fact, they give us enough information to completely describe the distribution of $\mathbf{y}_x$.

On the other hand, one can check that the target function for the hinge loss is *exactly* f^* . Thus, in a sufficiently rich hypothesis space—or equivalently, for an appropriately chosen kernel—the SVM classifier will converge to the simplest function (in terms of $\mathcal{R}$) that correctly classifies the data. This extends the geometric interpretation of SVM—for linear classification, the empirical risk is minimized by any function whose margins lie between the support vectors, and the simplest of these is the max-margin classifier.^[26]

Properties

SVMs belong to a family of generalized linear classifiers and can be interpreted as an extension of the perceptron.^[27] They can also be considered a special case of Tikhonov regularization. A special property is that they simultaneously minimize the empirical *classification error* and maximize the *geometric margin*; hence they are also known as **maximum margin classifiers**.

A comparison of the SVM to other classifiers has been made by Meyer, Leisch and Hornik.^[28]

Parameter selection

The effectiveness of SVM depends on the selection of kernel, the kernel's parameters, and soft margin parameter λ . A common choice is a Gaussian kernel, which has a single parameter γ . The best combination of λ and γ is often selected by a grid search with exponentially growing sequences of λ and γ , for example, $\lambda \in \{2^{-5}, 2^{-3}, \dots, 2^{13}, 2^{15}\}$; $\gamma \in \{2^{-15}, 2^{-13}, \dots, 2^1, 2^3\}$. Typically, each combination of parameter choices is checked using cross validation, and the parameters with best cross-validation accuracy are picked. Alternatively, recent work in Bayesian optimization can be used to select

λ and γ , often requiring the evaluation of far fewer parameter combinations than grid search. The final model, which is used for testing and for classifying new data, is then trained on the whole training set using the selected parameters.^[29]

Issues

Potential drawbacks of the SVM include the following aspects:

- Requires full labeling of input data
- Uncalibrated class membership probabilities—SVM stems from Vapnik's theory which avoids estimating probabilities on finite data
- The SVM is only directly applicable for two-class tasks. Therefore, algorithms that reduce the multi-class task to several binary problems have to be applied; see the multi-class SVM section.
- Parameters of a solved model are difficult to interpret.

Extensions

Multiclass SVM

Multiclass SVM aims to assign labels to instances by using support vector machines, where the labels are drawn from a finite set of several elements.

The dominant approach for doing so is to reduce the single multiclass problem into multiple binary classification problems.^[30] Common methods for such reduction include:^{[30][31]}

- Building binary classifiers that distinguish between one of the labels and the rest (*one-versus-all*) or between every pair of classes (*one-versus-one*). Classification of new instances for the one-versus-all case is done by a winner-takes-all strategy, in which the classifier with the highest-output function assigns the class (it is important that the output functions be calibrated to produce comparable scores). For the one-versus-one approach, classification is done by a max-wins voting strategy, in which every classifier assigns the instance to one of the two classes, then the vote for the assigned class is increased by one vote, and finally the class with the most votes determines the instance classification.
- Directed acyclic graph SVM (DAGSVM)^[32]
- Error-correcting output codes^[33]

Crammer and Singer proposed a multiclass SVM method which casts the multiclass classification problem into a single optimization problem, rather than decomposing it into multiple binary classification problems.^[34] See also Lee, Lin and Wahba^{[35][36]} and Van den Burg and Groenen.^[37]

Transductive support vector machines

Transductive support vector machines extend SVMs in that they could also treat partially labeled data in semi-supervised learning by following the principles of transduction. Here, in addition to the training set $\mathcal{D}$, the learner is also given a set

$$\mathcal{D}^* = \{\mathbf{x}_i^* \mid \mathbf{x}_i^* \in \mathbb{R}^p\}_{i=1}^k$$

of test examples to be classified. Formally, a transductive support vector machine is defined by the following primal optimization problem:^[38]

Minimize (in $\mathbf{w}, b, \mathbf{y}^*$)

$$\frac{1}{2} \|\mathbf{w}\|^2$$

subject to (for any $i = 1, \dots, n$ and any $j = 1, \dots, k$)

$$y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1,$$

$$y_j^*(\mathbf{w} \cdot \mathbf{x}_j^* - b) \geq 1,$$

and

$$y_j^* \in \{-1, 1\}.$$

Transductive support vector machines were introduced by Vladimir N. Vapnik in 1998.

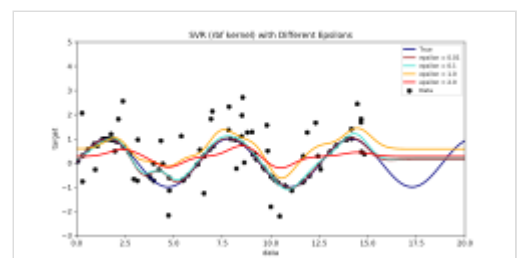
Structured SVM

Structured support-vector machine is an extension of the traditional SVM model. While the SVM model is primarily designed for binary classification, multiclass classification, and regression tasks, structured SVM broadens its application to handle general structured output labels, for example parse trees, classification with taxonomies, sequence alignment and many more.^[39]

Regression

A version of SVM for regression was proposed in 1996 by Vladimir N. Vapnik, Harris Drucker, Christopher J. C. Burges, Linda Kaufman and Alexander J. Smola.^[40] This method is called support vector regression (SVR). The model produced by support vector classification (as described above) depends only on a subset of the training data, because the cost function for building the model does not care about training points that lie beyond the margin. Analogously, the model produced by SVR depends only on a subset of the training data, because the cost function for building the model ignores any training data close to the model prediction. Another SVM version known as least-squares support vector machine (LS-SVM) has been proposed by Suykens and Vandewalle.^[41]

Training the original SVR means solving^[42]



Support vector regression (prediction) with different thresholds ϵ . As ϵ increases, the prediction becomes less sensitive to errors.

$$\begin{aligned} &\text{minimize } \frac{1}{2} \|w\|^2 \\ &\text{subject to } |y_i - \langle w, x_i \rangle - b| \leq \epsilon \end{aligned}$$

where x_i is a training sample with target value y_i . The inner product plus intercept $\langle w, x_i \rangle + b$ is the prediction for that sample, and ϵ is a free parameter that serves as a threshold: all predictions have to be within an ϵ range of the true predictions. Slack variables are usually added into the above to allow for errors and to allow approximation in the case the above problem is infeasible.

Bayesian SVM

In 2011 it was shown by Polson and Scott that the SVM admits a Bayesian interpretation through the technique of data augmentation.^[43] In this approach the SVM is viewed as a graphical model (where the parameters are connected via probability distributions). This extended view allows the application of Bayesian techniques to SVMs, such as flexible feature modeling, automatic hyperparameter tuning, and predictive uncertainty quantification. In 2017, a scalable version of the Bayesian SVM was developed by Florian Wenzel (<https://arxiv.org/search/stat?searchtype=author&query=Wenzel%2C+F>), enabling the application of Bayesian SVMs to big data.^[44] Florian Wenzel developed two different versions, a variational inference (VI) scheme for the Bayesian kernel support vector machine (SVM) and a stochastic version (SVI) for the linear Bayesian SVM.^[45]

Implementation

The parameters of the maximum-margin hyperplane are derived by solving the optimization. There exist several specialized algorithms for quickly solving the quadratic programming (QP) problem that arises from SVMs, mostly relying on heuristics for breaking the problem down into smaller, more manageable chunks.

Another approach is to use an interior-point method that uses Newton-like iterations to find a solution of the Karush–Kuhn–Tucker conditions of the primal and dual problems.^[46] Instead of solving a sequence of broken-down problems, this approach directly solves the problem altogether. To avoid solving a linear system involving the large kernel matrix, a low-rank approximation to the matrix is often used in the kernel trick.

Another common method is Platt's sequential minimal optimization (SMO) algorithm, which breaks the problem down into 2-dimensional sub-problems that are solved analytically, eliminating the need for a numerical optimization algorithm and matrix storage. This algorithm is conceptually simple, easy to implement, generally faster, and has better scaling properties for difficult SVM problems.^[47]

The special case of linear support vector machines can be solved more efficiently by the same kind of algorithms used to optimize its close cousin, logistic regression; this class of algorithms includes sub-gradient descent (e.g., PEGASOS^[48]) and coordinate descent (e.g., LIBLINEAR^[49]). LIBLINEAR has some attractive training-time properties. Each convergence iteration takes time linear in the time taken to read the train data, and the iterations also have a Q-linear convergence property, making the algorithm extremely fast.

The general kernel SVMs can also be solved more efficiently using sub-gradient descent (e.g. P-packSVM^[50]), especially when parallelization is allowed.

Kernel SVMs are available in many machine-learning toolkits, including [LIBSVM](http://support.sas.com/documentation/cdl/en/whatsnew/64209/HTML/default/viewer.htm#emdocwhatsnew71.htm), [MATLAB](http://support.sas.com/documentation/cdl/en/whatsnew/64209/HTML/default/viewer.htm#emdocwhatsnew71.htm), [SAS](http://support.sas.com/documentation/cdl/en/whatsnew/64209/HTML/default/viewer.htm#emdocwhatsnew71.htm) (<http://support.sas.com/documentation/cdl/en/whatsnew/64209/HTML/default/viewer.htm#emdocwhatsnew71.htm>), [SVMlight](https://cran.r-project.org/package=kernlab), [kernlab](https://cran.r-project.org/package=kernlab) (<https://cran.r-project.org/package=kernlab>), [scikit-learn](https://scikit-learn.org/), [Shogun](http://shogun.mfai.it/), [Weka](http://weka.sourceforge.net/), [Shark](http://image.diku.dk/shark/) (<http://image.diku.dk/shark/>), [JKernelMachines](https://mloss.org/software/view/409/) (<https://mloss.org/software/view/409/>), [OpenCV](http://opencv.org/) and others.

Preprocessing of data (standardization) is highly recommended to enhance accuracy of classification.^[51] There are a few methods of standardization, such as min-max, normalization by decimal scaling, Z-score.^[52] Subtraction of mean and division by variance of each feature is usually used for SVM.^[53]

See also

- [In situ adaptive tabulation](#)
- [Kernel machines](#)
- [Fisher kernel](#)
- [Platt scaling](#)
- [Polynomial kernel](#)
- [Predictive analytics](#)
- [Regularization perspectives on support vector machines](#)
- [Relevance vector machine](#), a probabilistic sparse-kernel model identical in functional form to SVM
- [Sequential minimal optimization](#)
- [Space mapping](#)
- [Winnow \(algorithm\)](#)
- [Radial basis function network](#)

References

1. Cortes, Corinna; Vapnik, Vladimir (1995). "Support-vector networks" (http://image.diku.dk/imagedoc/material/cortes_vapnik95.pdf) (PDF). *Machine Learning*. **20** (3): 273–297. doi:10.1007/BF00994018 (<https://doi.org/10.1007%2FBF00994018>). S2CID 206787478 (<https://api.semanticscholar.org/CorpusID:206787478>).
2. Vapnik, Vladimir N. (1997). "The Support Vector method" (<https://link.springer.com/chapter/10.1007/BFb0020166>). In Gerstner, Wulfram; Germond, Alain; Hasler, Martin; Nicoud, Jean-Daniel (eds.). *Artificial Neural Networks — ICANN'97*. Lecture Notes in Computer Science. Vol. 1327. Berlin, Heidelberg: Springer. pp. 261–271. doi:10.1007/BFb0020166 (<https://doi.org/10.1007%2FBFb0020166>). ISBN 978-3-540-69620-9.
3. Awad, Mariette; Khanna, Rahul (2015). "Support Vector Machines for Classification". *Efficient Learning Machines*. Apress. pp. 39–66. doi:10.1007/978-1-4302-5990-9_3 (https://doi.org/10.1007%2F978-1-4302-5990-9_3). ISBN 978-1-4302-5990-9.
4. Ben-Hur, Asa; Horn, David; Siegelmann, Hava; Vapnik, Vladimir N. "Support vector clustering" (2001);". *Journal of Machine Learning Research*. **2**: 125–137.

5. Huang, H. H.; Xu, T.; Yang, J. (2014). "Comparing logistic regression, support vector machines, and permanental classification methods in predicting hypertension" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4143639>). *BMC Proceedings*. **8** (Suppl 1): S96. doi:10.1186/1753-6561-8-S1-S96 (<https://doi.org/10.1186%2F1753-6561-8-S1-S96>). PMC 4143639 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4143639>). PMID 25519351 (<https://pubmed.ncbi.nlm.nih.gov/25519351>).
6. Oppen, M; Kinzel, W; Kleinz, J; Nehl, R (1990). "On the ability of the optimal perceptron to generalise". *Journal of Physics A: Mathematical and General*. **23** (11): L581. Bibcode:1990JPhA...23L.581O (<https://ui.adsabs.harvard.edu/abs/1990JPhA...23L.581O>). doi:10.1088/0305-4470/23/11/012 (<https://doi.org/10.1088%2F0305-4470%2F23%2F11%2F012>).
7. "1.4. Support Vector Machines — scikit-learn 0.20.2 documentation" (<http://scikit-learn.org/stable/modules/svm.html>). Archived (<https://web.archive.org/web/20171108151644/http://scikit-learn.org/stable/modules/svm.html>) from the original on 2017-11-08. Retrieved 2017-11-08.
8. Hastie, Trevor; Tibshirani, Robert; Friedman, Jerome (2008). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (https://hastie.su.domains/ElemStatLearn/printings/ESLII_print12_toc.pdf.download.html) (Second ed.). New York: Springer. p. 134.
9. Boser, Bernhard E.; Guyon, Isabelle M.; Vapnik, Vladimir N. (1992). "A training algorithm for optimal margin classifiers" (<http://www.clopinet.com/isabelle/Papers/colt92.ps.Z>). *Proceedings of the fifth annual workshop on Computational learning theory – COLT '92*. p. 144. doi:10.1145/130385.130401 (<https://doi.org/10.1145%2F130385.130401>). ISBN 978-0-89791-497-0. S2CID 207165665 (<https://api.semanticscholar.org/CorpusID:207165665>).
10. Press, William H.; Teukolsky, Saul A.; Vetterling, William T.; Flannery, Brian P. (2007). "Section 16.5. Support Vector Machines" (<http://apps.nrbook.com/empanel/index.html#pg=883>). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). New York: Cambridge University Press. ISBN 978-0-521-88068-8. Archived (<https://web.archive.org/web/20110811154417/http://apps.nrbook.com/empanel/index.html#pg=883>) from the original on 2011-08-11.
11. Joachims, Thorsten (1998). "Text categorization with Support Vector Machines: Learning with many relevant features". *Machine Learning: ECML-98. Lecture Notes in Computer Science*. Vol. 1398. Springer. pp. 137–142. doi:10.1007/BFb0026683 (<https://doi.org/10.1007%2FBFb0026683>). ISBN 978-3-540-64417-0.
12. Pradhan, Sameer S.; et al. (2 May 2004). *Shallow Semantic Parsing using Support Vector Machines* (<http://www.aclweb.org/anthology/N04-1030>). Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004. Association for Computational Linguistics. pp. 233–240.
13. Vapnik, Vladimir N.: Invited Speaker. IPMU Information Processing and Management 2014).
14. Barghout, Lauren (2015). "Spatial-Taxon Information Granules as Used in Iterative Fuzzy-Decision-Making for Image Segmentation" (<https://web.archive.org/web/20180108120422/https://pdfs.semanticscholar.org/917f/15d33d32062bffe6401eee9fe71d16d6a84.pdf>) (PDF). *Granular Computing and Decision-Making*. Studies in Big Data. Vol. 10. pp. 285–318. doi:10.1007/978-3-319-16829-6_12 (https://doi.org/10.1007%2F978-3-319-16829-6_12). ISBN 978-3-319-16828-9. S2CID 4154772 (<https://api.semanticscholar.org/CorpusID:4154772>). Archived from the original (<https://pdfs.semanticscholar.org/917f/15d33d32062bffe6401eee9fe71d16d6a84.pdf>) (PDF) on 2018-01-08. Retrieved 2018-01-08.
15. A. Maity (2016). "Supervised Classification of RADARSAT-2 Polarimetric Data for Different Land Features". arXiv:1608.00501 (<https://arxiv.org/abs/1608.00501>) [cs.CV (<https://arxiv.org/archive/cs.CV>)].

16. DeCoste, Dennis (2002). "Training Invariant Support Vector Machines" (<https://people.eecs.berkeley.edu/~malik/cs294/decoste-scholkopf.pdf>) (PDF). *Machine Learning*. **46** (1–3): 161–190. Bibcode:2002MLear..46..161D (<https://ui.adsabs.harvard.edu/abs/2002MLear..46..161D>). doi:10.1023/A:1012454411458 (<https://doi.org/10.1023%2FA%3A1012454411458>). S2CID 85843 (<https://api.semanticscholar.org/CorpusID:85843>).
17. Maitra, D. S.; Bhattacharya, U.; Parui, S. K. (August 2015). "CNN based common approach to handwritten character recognition of multiple scripts". *2015 13th International Conference on Document Analysis and Recognition (ICDAR)*. pp. 1021–1025. doi:10.1109/ICDAR.2015.7333916 (<https://doi.org/10.1109%2FICDAR.2015.7333916>). ISBN 978-1-4799-1805-8. S2CID 25739012 (<https://api.semanticscholar.org/CorpusID:25739012>).
18. Gaonkar, B.; Davatzikos, C. (2013). "Analytic estimation of statistical significance maps for support vector machine based multi-variate image analysis and classification" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3767485>). *NeuroImage*. **78**: 270–283. doi:10.1016/j.neuroimage.2013.03.066 (<https://doi.org/10.1016%2Fj.neuroimage.2013.03.066>). PMC 3767485 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3767485>). PMID 23583748 (<https://pubmed.ncbi.nlm.nih.gov/23583748>).
19. Cuingnet, Rémi; Rosso, Charlotte; Chupin, Marie; Lehericy, Stéphane; Dormont, Didier; Benali, Habib; Samson, Yves; Colliot, Olivier (2011). "Spatial regularization of SVM for the detection of diffusion alterations associated with stroke outcome" (https://web.archive.org/web/20181222172844/http://www.aramislab.fr/perso/colliot/files/media2011_remi_published.pdf) (PDF). *Medical Image Analysis*. **15** (5): 729–737. doi:10.1016/j.media.2011.05.007 (<https://doi.org/10.1016%2Fj.media.2011.05.007>). PMID 21752695 (<https://pubmed.ncbi.nlm.nih.gov/21752695>). Archived from the original (http://www.aramislab.fr/perso/colliot/files/media2011_remi_published.pdf) (PDF) on 2018-12-22. Retrieved 2018-01-08.
20. Statnikov, Alexander; Hardin, Douglas; & Aliferis, Constantin; (2006); "Using SVM weight-based methods to identify causally relevant and non-causally relevant variables" (http://www.ccdlab.org/paper-pdfs/NIPS_2006.pdf), *Sign*, 1, 4.
21. "Why is the SVM margin equal to $\frac{2}{\|\mathbf{w}\|}$ " (<https://math.stackexchange.com/q/1305925/168764>). *Mathematics Stack Exchange*. 30 May 2015.
22. Aizerman, Mark A.; Braverman, Emmanuel M. & Rozonoer, Lev I. (1964). "Theoretical foundations of the potential function method in pattern recognition learning". *Automation and Remote Control*. **25**: 821–837.
23. Jin, Chi; Wang, Liwei (2012). *Dimensionality dependent PAC-Bayes margin bound* (<http://papers.nips.cc/paper/4500-dimensionality-dependent-pac-bayes-margin-bound>). Advances in Neural Information Processing Systems. Archived (<https://web.archive.org/web/20150402185336/http://papers.nips.cc/paper/4500-dimensionality-dependent-pac-bayes-margin-bound>) from the original on 2015-04-02.
24. Shalev-Shwartz, Shai; Singer, Yoram; Srebro, Nathan; Cotter, Andrew (2010-10-16). "Pegasos: primal estimated sub-gradient solver for SVM". *Mathematical Programming*. **127** (1): 3–30. doi:10.1007/s10107-010-0420-4 (<https://doi.org/10.1007%2Fs10107-010-0420-4>). ISSN 0025-5610 (<https://search.worldcat.org/issn/0025-5610>). S2CID 53306004 (<https://api.semanticscholar.org/CorpusID:53306004>).
25. Hsieh, Cho-Jui; Chang, Kai-Wei; Lin, Chih-Jen; Keerthi, S. Sathiya; Sundararajan, S. (2008-01-01). "A dual coordinate descent method for large-scale linear SVM". *Proceedings of the 25th international conference on Machine learning - ICML '08*. New York, NY, USA: ACM. pp. 408–415. doi:10.1145/1390156.1390208 (<https://doi.org/10.1145%2F1390156.1390208>). ISBN 978-1-60558-205-4. S2CID 7880266 (<https://api.semanticscholar.org/CorpusID:7880266>).

26. Rosasco, Lorenzo; De Vito, Ernesto; Caponnetto, Andrea; Piana, Michele; Verri, Alessandro (2004-05-01). "Are Loss Functions All the Same?" (<https://ieeexplore.ieee.org/document/6789841>). *Neural Computation*. **16** (5): 1063–1076. doi:10.1162/089976604773135104 (<https://doi.org/10.1162%2F089976604773135104>). hdl:11380/4590 (<https://hdl.handle.net/11380%2F4590>). ISSN 0899-7667 (<https://search.worldcat.org/issn/0899-7667>). PMID 15070510 (<https://pubmed.ncbi.nlm.nih.gov/15070510>). S2CID 11845688 (<https://api.semanticscholar.org/CorpusID:11845688>).
27. R. Collobert and S. Bengio (2004). Links between Perceptrons, MLPs and SVMs. Proc. Int'l Conf. on Machine Learning (ICML).
28. Meyer, David; Leisch, Friedrich; Hornik, Kurt (September 2003). "The support vector machine under test". *Neurocomputing*. **55** (1–2): 169–186. doi:10.1016/S0925-2312(03)00431-4 (<https://doi.org/10.1016%2FS0925-2312%2803%2900431-4>).
29. Hsu, Chih-Wei; Chang, Chih-Chung & Lin, Chih-Jen (2003). *A Practical Guide to Support Vector Classification* (<http://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>) (PDF) (Technical report). Department of Computer Science and Information Engineering, National Taiwan University. Archived (<https://web.archive.org/web/20130625201224/http://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>) (PDF) from the original on 2013-06-25.
30. Duan, Kai-Bo; Keerthi, S. Sathya (2005). "Which Is the Best Multiclass SVM Method? An Empirical Study" (<https://web.archive.org/web/20130503183745/http://www.cs.iastate.edu/~honavar/multiclass-svm2.pdf>) (PDF). *Multiple Classifier Systems*. LNCS. Vol. 3541. pp. 278–285. doi:10.1007/11494683_28 (https://doi.org/10.1007%2F11494683_28). ISBN 978-3-540-26306-7. Archived from the original (<https://www.cs.iastate.edu/~honavar/multiclass-svm2.pdf>) (PDF) on 2013-05-03. Retrieved 2019-07-18.
31. Hsu, Chih-Wei & Lin, Chih-Jen (2002). "A Comparison of Methods for Multiclass Support Vector Machines" (<https://web.archive.org/web/20130503183743/http://www.cs.iastate.edu/~honavar/multiclass-svm.pdf>) (PDF). *IEEE Transactions on Neural Networks*. **13** (2): 415–25. Bibcode:2002ITNN...13..415H (<https://ui.adsabs.harvard.edu/abs/2002ITNN...13..415H>). doi:10.1109/72.991427 (<https://doi.org/10.1109%2F72.991427>). PMID 18244442 (<https://pubmed.ncbi.nlm.nih.gov/18244442>). Archived from the original (<http://www.cs.iastate.edu/~honavar/multiclass-svm.pdf>) (PDF) on 2013-05-03. Retrieved 2018-01-08.
32. Platt, John; Cristianini, Nello; Shawe-Taylor, John (2000). "Large margin DAGs for multiclass classification" (<http://www.wisdom.weizmann.ac.il/~bagon/CVspring07/files/DAGSVM.pdf>) (PDF). In Solla, Sara A.; Leen, Todd K.; Müller, Klaus-Robert (eds.). *Advances in Neural Information Processing Systems*. MIT Press. pp. 547–553. Archived (<https://web.archive.org/web/20120616221540/http://www.wisdom.weizmann.ac.il/~bagon/CVspring07/files/DAGSVM.pdf>) (PDF) from the original on 2012-06-16.
33. Dietterich, Thomas G.; Bakiri, Ghulum (1995). "Solving Multiclass Learning Problems via Error-Correcting Output Codes" (<http://www.jair.org/media/105/live-105-1426-jair.pdf>) (PDF). *Journal of Artificial Intelligence Research*. **2**: 263–286. arXiv:cs/9501101 (<https://arxiv.org/abs/cs/9501101>). Bibcode:1995cs.....1101D (<https://ui.adsabs.harvard.edu/abs/1995cs.....1101D>). doi:10.1613/jair.105 (<https://doi.org/10.1613%2Fjair.105>). S2CID 47109072 (<https://api.semanticscholar.org/CorpusID:47109072>). Archived (<https://web.archive.org/web/20130509061344/http://www.jair.org/media/105/live-105-1426-jair.pdf>) (PDF) from the original on 2013-05-09.
34. Crammer, Koby & Singer, Yoram (2001). "On the Algorithmic Implementation of Multiclass Kernel-based Vector Machines" (<http://jmlr.csail.mit.edu/papers/volume2/crammer01a/crammer01a.pdf>) (PDF). *Journal of Machine Learning Research*. **2**: 265–292. Archived (<https://web.archive.org/web/20150829102651/http://jmlr.csail.mit.edu/papers/volume2/crammer01a/crammer01a.pdf>) (PDF) from the original on 2015-08-29.
35. Lee, Yoonkyung; Lin, Yi & Wahba, Grace (2001). "Multicategory Support Vector Machines" (<https://web.archive.org/web/20130617093314/http://www.interfacesymposia.org/I01/I2001Proceedings/YLee/YLee.pdf>) (PDF). *Computing Science and Statistics*. **33**. Archived from the original on 2013-06-17.

36. Lee, Yoonkyung; Lin, Yi; Wahba, Grace (2004). "Multicategory Support Vector Machines". *Journal of the American Statistical Association*. **99** (465): 67–81. doi:10.1198/016214504000000098 (<https://doi.org/10.1198%2F016214504000000098>). S2CID 7066611 (<https://api.semanticscholar.org/CorpusID:7066611>).
37. Van den Burg, Gerrit J. J. & Groenen, Patrick J. F. (2016). "GenSVM: A Generalized Multiclass Support Vector Machine" (<http://jmlr.org/papers/volume17/14-526/14-526.pdf>) (PDF). *Journal of Machine Learning Research*. **17** (224): 1–42.
38. Joachims, Thorsten. *Transductive Inference for Text Classification using Support Vector Machines* (<http://www1.cs.columbia.edu/~dplewis/candidacy/joachims99transductive.pdf>) (PDF). Proceedings of the 1999 International Conference on Machine Learning (ICML 1999). pp. 200–209.
39. "Support Vector Machine Learning for Interdependent and Structured Output Spaces" (http://www.cs.cornell.edu/people/tj/publications/tsochantaridis_etal_04a.pdf) (PDF). www.cs.cornell.edu.
40. Drucker, Harris; Burges, Christ. C.; Kaufman, Linda; Smola, Alexander J.; and Vapnik, Vladimir N. (1997); "Support Vector Regression Machines" (<http://papers.nips.cc/paper/1238-support-vector-regression-machines.pdf>), in *Advances in Neural Information Processing Systems 9, NIPS 1996*, 155–161, MIT Press.
41. Suykens, Johan A. K.; Vandewalle, Joos P. L.; "Least squares support vector machine classifiers" (https://lirias.kuleuven.be/bitstream/123456789/218716/2/Suykens_NeurProcLett.pdf), *Neural Processing Letters*, vol. 9, no. 3, Jun. 1999, pp. 293–300.
42. Smola, Alex J.; Schölkopf, Bernhard (2004). "A tutorial on support vector regression" (<http://eprints.pascal-network.org/archive/00000856/01/fulltext.pdf>) (PDF). *Statistics and Computing*. **14** (3): 199–222. Bibcode:2004StCom..14..199S (<https://ui.adsabs.harvard.edu/abs/2004StCom..14..199S>). doi:10.1023/B:STCO.0000035301.49549.88 (<https://doi.org/10.1023%2FB%3ASTCO.0000035301.49549.88>). S2CID 15475 (<https://api.semanticscholar.org/CorpusID:15475>). Archived (<https://web.archive.org/web/20120131193522/http://eprints.pascal-network.org/archive/00000856/01/fulltext.pdf>) (PDF) from the original on 2012-01-31.
43. Polson, Nicholas G.; Scott, Steven L. (2011). "Data Augmentation for Support Vector Machines" (<https://doi.org/10.1214%2F11-BA601>). *Bayesian Analysis*. **6** (1): 1–23. doi:10.1214/11-BA601 (<https://doi.org/10.1214%2F11-BA601>).
44. Wenzel, Florian; Galy-Fajou, Theo; Deutsch, Matthäus; Kloft, Marius (2017). "Bayesian Nonlinear Support Vector Machines for Big Data". *Machine Learning and Knowledge Discovery in Databases. Lecture Notes in Computer Science*. Vol. 10534. pp. 307–322. arXiv:1707.05532 (<https://arxiv.org/abs/1707.05532>). Bibcode:2017arXiv170705532W (<http://ui.adsabs.harvard.edu/abs/2017arXiv170705532W>). doi:10.1007/978-3-319-71249-9_19 (https://doi.org/10.1007%2F978-3-319-71249-9_19). ISBN 978-3-319-71248-2. S2CID 4018290 (<https://api.semanticscholar.org/CorpusID:4018290>).
45. Florian Wenzel; Matthäus Deutsch; Théo Galy-Fajou; Marius Kloft; "Scalable Approximate Inference for the Bayesian Nonlinear Support Vector Machine" (<http://approximateinference.org/accepted/WenzelEtAl2016.pdf>)
46. Ferris, Michael C.; Munson, Todd S. (2002). "Interior-Point Methods for Massive Support Vector Machines" (<http://www.cs.wisc.edu/~ferris/papers/siopt-svm.pdf>) (PDF). *SIAM Journal on Optimization*. **13** (3): 783–804. doi:10.1137/S1052623400374379 (<https://doi.org/10.1137%2FS1052623400374379>). S2CID 13563302 (<https://api.semanticscholar.org/CorpusID:13563302>). Archived (<https://web.archive.org/web/20081204224416/http://www.cs.wisc.edu/~ferris/papers/siopt-svm.pdf>) (PDF) from the original on 2008-12-04.
47. Platt, John C. (1998). *Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines* (<http://research.microsoft.com/pubs/69644/tr-98-14.pdf>) (PDF). NIPS. Archived (<https://web.archive.org/web/20150702075055/http://research.microsoft.com/pubs/69644/tr-98-14.pdf>) (PDF) from the original on 2015-07-02.

48. Shalev-Shwartz, Shai; Singer, Yoram; Srebro, Nathan (2007). *Pegasos: Primal Estimated sub-GrAdient SOLver for SVM* (<http://ttic.uchicago.edu/~shai/papers/ShalevSiSr07.pdf>) (PDF). ICML. Archived (<https://web.archive.org/web/20131215051700/http://ttic.uchicago.edu/~shai/papers/ShalevSiSr07.pdf>) (PDF) from the original on 2013-12-15.
49. Fan, Rong-En; Chang, Kai-Wei; Hsieh, Cho-Jui; Wang, Xiang-Rui; Lin, Chih-Jen (2008). "LIBLINEAR: A library for large linear classification" (<https://www.csie.ntu.edu.tw/~cjlin/paper/s/liblinear.pdf>) (PDF). *Journal of Machine Learning Research*. **9**: 1871–1874.
50. Allen Zhu, Zeyuan; Chen, Weizhu; Wang, Gang; Zhu, Chenguang; Chen, Zheng (2009). *P-packSVM: Parallel Primal grAdient desCent Kernel SVM* (<http://people.csail.mit.edu/zeyuan/paper/2009-ICDM-Parallel.pdf>) (PDF). ICDM. Archived (<https://web.archive.org/web/20140407095709/http://people.csail.mit.edu/zeyuan/paper/2009-ICDM-Parallel.pdf>) (PDF) from the original on 2014-04-07.
51. Fan, Rong-En; Chang, Kai-Wei; Hsieh, Cho-Jui; Wang, Xiang-Rui; Lin, Chih-Jen (2008). "LIBLINEAR: A library for large linear classification". *Journal of Machine Learning Research*. **9** (Aug): 1871–1874.
52. Mohamad, Ismail; Usman, Dauda (2013-09-01). "Standardization and Its Effects on K-Means Clustering Algorithm" (<https://doi.org/10.19026/rjaset.6.3638>). *Research Journal of Applied Sciences, Engineering and Technology*. **6** (17): 3299–3303. doi:10.19026/rjaset.6.3638 (<https://doi.org/10.19026/rjaset.6.3638>).
53. Fennell, Peter; Zuo, Zhiya; Lerman, Kristina (2019-12-01). "Predicting and explaining behavioral data with structured feature space decomposition" (<https://doi.org/10.1140%2Fepjds%2Fs13688-019-0201-0>). *EPJ Data Science*. **8** 23. arXiv:1810.09841 (<https://arxiv.org/abs/1810.09841>). doi:10.1140/epjds/s13688-019-0201-0 (<https://doi.org/10.1140%2Fepjds%2Fs13688-019-0201-0>).

Further reading

- Bennett, Kristin P.; Campbell, Colin (2000). "Support Vector Machines: Hype or Hallelujah?" (http://kdd.org/exploration_files/bennett.pdf) (PDF). *SIGKDD Explorations*. **2** (2): 1–13. doi:10.1145/380995.380999 (<https://doi.org/10.1145%2F380995.380999>). S2CID 207753020 (<https://api.semanticscholar.org/CorpusID:207753020>).
- Cristianini, Nello; Shawe-Taylor, John (2000). *An Introduction to Support Vector Machines and other kernel-based learning methods*. Cambridge University Press. ISBN 0-521-78019-5.
- Fradkin, Dmitriy; Muchnik, Ilya (2006). "Support Vector Machines for Classification" (<http://pa-ul.rutgers.edu/~dfradkin/papers/svm.pdf>) (PDF). In Abello, J.; Carmode, G. (eds.). *Discrete Methods in Epidemiology*. DIMACS Series in Discrete Mathematics and Theoretical Computer Science. Vol. 70. pp. 13–20.
- Joachims, Thorsten (1998). "Text categorization with Support Vector Machines: Learning with many relevant features". In Nédellec, Claire; Rouveirol, Céline (eds.). *Machine Learning: ECML-98*. Lecture Notes in Computer Science. Vol. 1398. Berlin, Heidelberg: Springer. pp. 137–142. doi:10.1007/BFb0026683 (<https://doi.org/10.1007%2FBFb0026683>). ISBN 978-3-540-64417-0. S2CID 2427083 (<https://api.semanticscholar.org/CorpusID:2427083>).
- Ivanciuc, Ovidiu (2007). "Applications of Support Vector Machines in Chemistry" (http://www.ivanciuc.org/Files/Reprint/Ivanciuc_SVM_CCR_2007_23_291.pdf) (PDF). *Reviews in Computational Chemistry*. Vol. 23. pp. 291–400. doi:10.1002/9780470116449.ch6 (<https://doi.org/10.1002%2F9780470116449.ch6>). ISBN 978-0-470-11644-9.
- James, Gareth; Witten, Daniela; Hastie, Trevor; Tibshirani, Robert (2013). "Support Vector Machines" (<http://www-bcf.usc.edu/~gareth/ISL/ISLR%20Seventh%20Printing.pdf#page=34>

5) (PDF). *An Introduction to Statistical Learning: with Applications in R*. New York: Springer. pp. 337–372. ISBN 978-1-4614-7137-0.

- Schölkopf, Bernhard; Smola, Alexander J. (2002). *Learning with Kernels*. Cambridge, MA: MIT Press. ISBN 0-262-19475-9.
- Steinwart, Ingo; Christmann, Andreas (2008). *Support Vector Machines*. New York: Springer. ISBN 978-0-387-77241-7.
- Theodoridis, Sergios; Koutroumbas, Konstantinos (2009). *Pattern Recognition* (4th ed.). Academic Press. ISBN 978-1-59749-272-0.

External links

- [libsvm \(http://www.csie.ntu.edu.tw/~cjlin/libsvm/\)](http://www.csie.ntu.edu.tw/~cjlin/libsvm/), [LIBSVM](#) is a popular library of SVM learners
- [liblinear \(http://www.csie.ntu.edu.tw/~cjlin/liblinear/\)](http://www.csie.ntu.edu.tw/~cjlin/liblinear/) is a library for large linear classification including some SVMs
- [SVM light \(http://svmlight.joachims.org\)](http://svmlight.joachims.org) is a collection of software tools for learning and classification using SVM
- [SVMJS live demo \(http://cs.stanford.edu/people/karpathy/svmjs/demo/\)](http://cs.stanford.edu/people/karpathy/svmjs/demo/) Archived (<https://web.archive.org/web/20130505185215/http://cs.stanford.edu/people/karpathy/svmjs/demo/>) 2013-05-05 at the [Wayback Machine](#) is a GUI demo for [JavaScript](#) implementation of SVMs

Retrieved from "https://en.wikipedia.org/w/index.php?title=Support_vector_machine&oldid=1373878364"